

PLATFORMY TECHNOLOGICZNE LABORATORIUM 2_3

Benjamin Jurewicz
s198326

Marta Kociszewska
s198143

Lidia Zawrzykraj
s198257

1. OPIS PROGRAMU

Stosowanie wątków w celu zaimplementowania problemu producentów i konsumentów.

Korzyści używania wątków:

- równoległe przetwarzanie
- równoległe wykonywania zadań

W kontekście problemu producentów i konsumentów:

- przetwarzanie danych przez producentów
 - Producenci asynchronicznie generują dane, które są przekazywane do bufora (FIFO).
 - W przypadku, gdy bufor jest pełny, producent czeka na zwolnienie miejsca w buforze.
 - Przetwarzanie danych przez Producenta - oblicza dzielniki wygenerowanej liczby
- przetwarzanie danych przez konsumentów
 - Konsument pobiera dane z bufora i je wypisuje.
 - W przypadku, gdy bufor jest pusty, konsument czeka na pojawienie się danych w buforze.
- równoległe wykonywanie zadań
- synchronizacja dostępu do współdzielonych zasobów aby uniknąć problemów związanych z dostępem do nich przez wiele wątków

2. ZAKRES UŻYCIA WIELOWĄTKOWOŚCI I LOGIKI PRZETWARZANIA DANYCH W OPROGRAMOWANIU

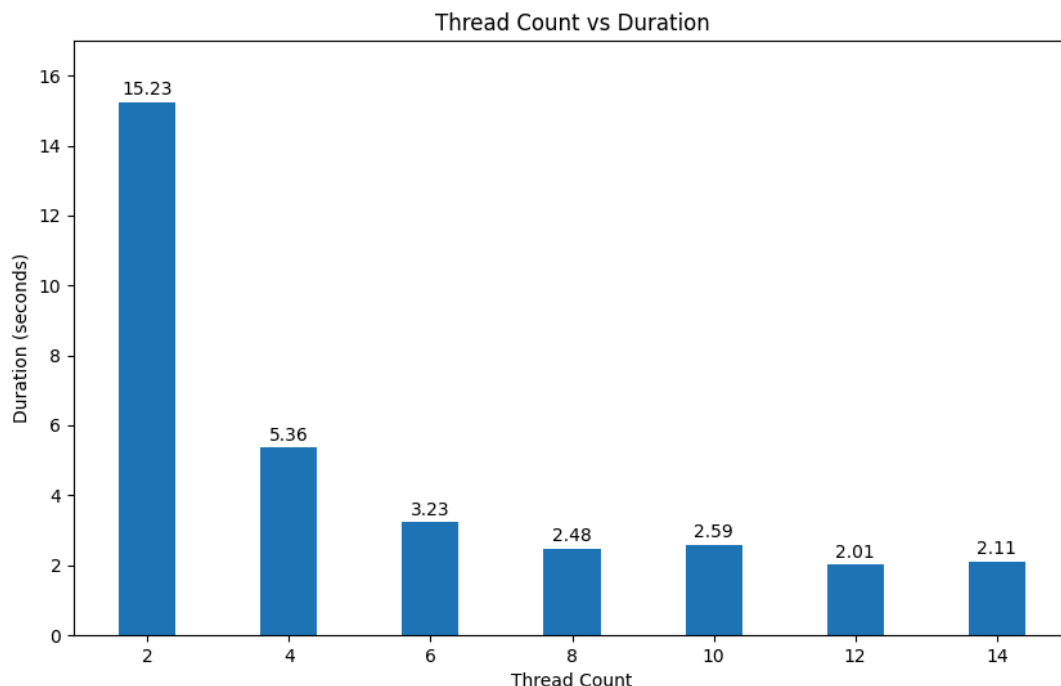
Użycie wielowątkowości

Wielowątkowość w oprogramowaniu pozwala na równoczesne wykonywanie wielu wątków, co umożliwia bardziej efektywne wykorzystanie zasobów CPU. W dostarczonym kodzie wielowątkowość jest zaimplementowana za pomocą klas *Producer* i *Consumer*, które działają w oddzielnych wątkach, aby obsługiwać produkcję i konsumpcję danych równocześnie. *BlockingQueue* jest używana do zarządzania komunikacją między tymi wątkami, zapewniając bezpieczne operacje bez potrzeby zarządzania wątkami w sposób jawny.

Logika przetwarzania danych

Logika przetwarzania danych obejmuje generowanie liczb i obliczanie ich dzielników. Klasa *Producer* generuje liczby za pomocą klasy *NumberGenerator* i oblicza ich dzielniki. Wyniki są enkapsulowane w obiektach *Answer* i umieszczane w *BlockingQueue*. Klasa *Consumer* pobiera te obiekty *Answer* z kolejki i przetwarza je, co w tym przypadku polega na drukowaniu wyników.

3. ANALIZA DANYCH



Rysunek 1: Czas trwania programu w zależności od ilości wątków

1. Zwiększenie liczby wątków znacząco skraca czas wykonania – przy najmniejszej liczbie wątków (2) czas wykonania wynosi około 15,23 sekundy, natomiast przy większej liczbie wątków czas ten maleje.
2. Największy spadek czasu wykonania następuje dla początkowych wartości liczby wątków – przejście od 2 do 4 wątków powoduje drastyczny spadek czasu wykonania z 15,23 do 5,36 sekundy. Kolejne zwiększanie liczby wątków nadal skraca czas, ale efektywność wzrostu liczby wątków maleje.
3. Dalsze zwiększanie liczby wątków przestaje przynosić znaczące korzyści – po osiągnięciu 8-10 wątków czas wykonania oscyluje wokół 2 sekund i dalsze zwiększanie liczby wątków nie powoduje znaczącego skrócenia czasu (np. dla 12 wątków czas wynosi 2,01 s, a dla 14 wątków nawet nieznacznie rośnie do 2,11 s).
4. Możliwe ograniczenia sprzętowe lub narzut zarządzania wątkami – niewielkie różnice w czasie wykonania między 8, 10, 12 i 14 wątkami mogą wskazywać na osiągnięcie limitu optymalnej równoległości dla tego zadania. Dalsze dodawanie wątków może powodować narzut związany z zarządzaniem wielowątkowością, co skutkuje minimalnym wzrostem czasu wykonania.

Poprawność Obliczeń

Dzielniki liczby: 922337203685477573

Program:

Divisor Count: 8

Divisors: 1, 7, 89, 623, 1480477052464651, 10363339367252557, 131762457669353939, 922337203685477573

Poprawne:

1 | 7 | 89 | 623 | 1480477052464651 | 10363339367252557 | 131762457669353939 | 922337203685477573 (8 divisors)

(source: www.wolframalpha.com)

Uzyskane wyniki pokrywają się z danymi referencyjnymi z Wolfram Alpha, co potwierdza poprawność działania programu. Program prawidłowo znajduje wszystkie 8 dzielników liczby 922337203685477573, co świadczy o jego dokładności i poprawności obliczeń.

4. WNIOSKI Z ANALIZY UZYSKANYCH WYNIKÓW

Obserwacje

1. **Efektywne wykorzystanie CPU:** Wielowątkowość pozwala na lepsze wykorzystanie CPU poprzez równoczesne wykonywanie zadań producenta i konsumenta.
2. **Bezpieczeństwo wątków:** Użycie *BlockingQueue* zapewnia bezpieczną komunikację między wątkami producenta i konsumenta.
3. **Skalowalność:** System można skalować, dodając więcej wątków producenta lub konsumenta, aby obsłużyć większe ilości danych.

Ograniczenia

1. **Ograniczenia sprzętowe:** Zyski wydajności z wielowątkowości są ograniczone przez liczbę dostępnych rdzeni CPU. W systemach z mniejszą liczbą rdzeni korzyści z wielowątkowości mogą być mniej wyraźne.
2. **Koszt przełączania kontekstu:** Nadmierne przełączanie kontekstu między wątkami może prowadzić do degradacji wydajności, zwłaszcza jeśli liczba wątków przekracza liczbę rdzeni CPU.
3. **Zużycie pamięci:** Każdy wątek zużywa pamięć, a tworzenie zbyt wielu wątków może prowadzić do zwiększonego zużycia pamięci i potencjalnego wyczerpania zasobów systemowych.

Zastosowanie Wielowątkowości

Wielowątkowość jest szczególnie przydatna w scenariuszach, gdzie zadania mogą działać równoległe i nie zależą od siebie nawzajem. Wielowątkowość jest dobrze dopasowana do niezależnych zadań generowania liczb i obliczania dzielników. Jednakże, dla zadań wymagających znacznej komunikacji między wątkami lub synchronizacji, koszty zarządzania wątkami mogą przewyższać korzyści.

Podsumowanie

Chociaż wielowątkowość może znacznie poprawić wydajność i efektywność w aplikacjach przetwarzania danych, ważne jest uwzględnienie ograniczeń sprzętowych i charakteru zadań, które mają być równoległe, aby osiągnąć optymalne wyniki.